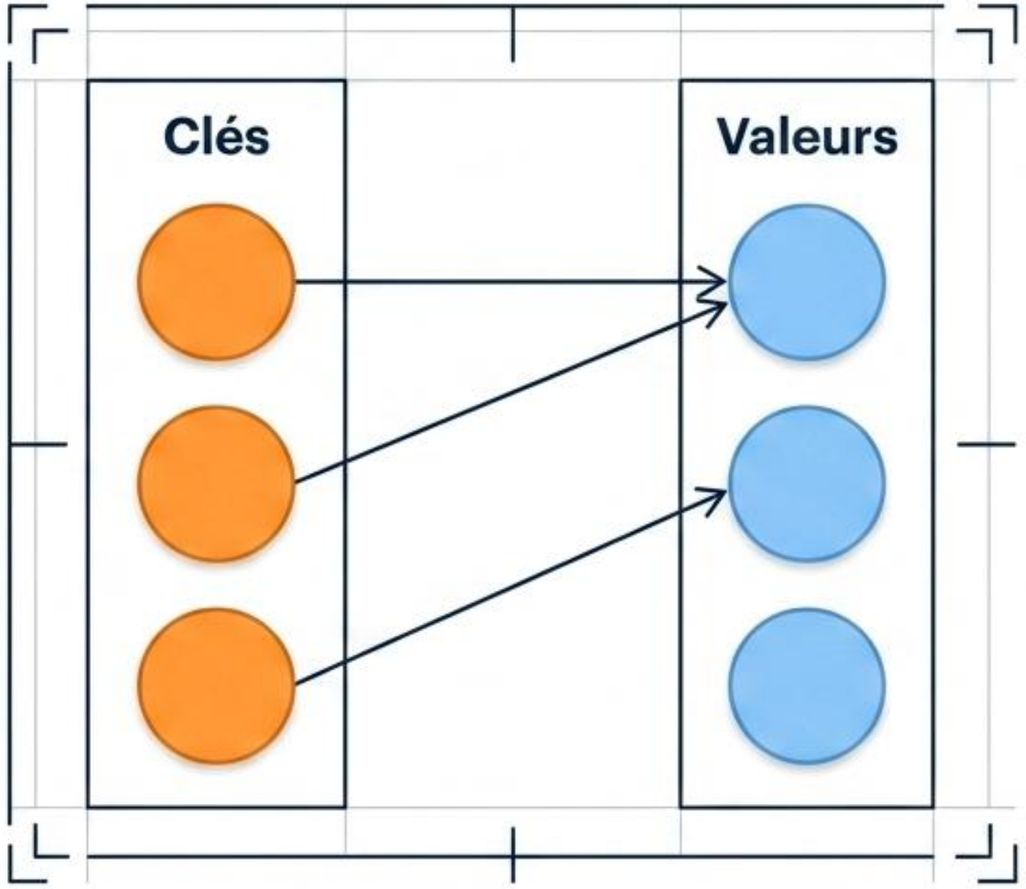
L'Écosystème Map en Java

Architecture, Implémentations et Cas d'Usage Pratiques pour la Programmation Orientée Objet.



L'Anatomie d'un Dictionnaire

Clés Uniques : Une Map ne peut pas contenir de clés dupliquées. Chaque clé est unique.

Association : Chaque clé mappe vers exactement une valeur.

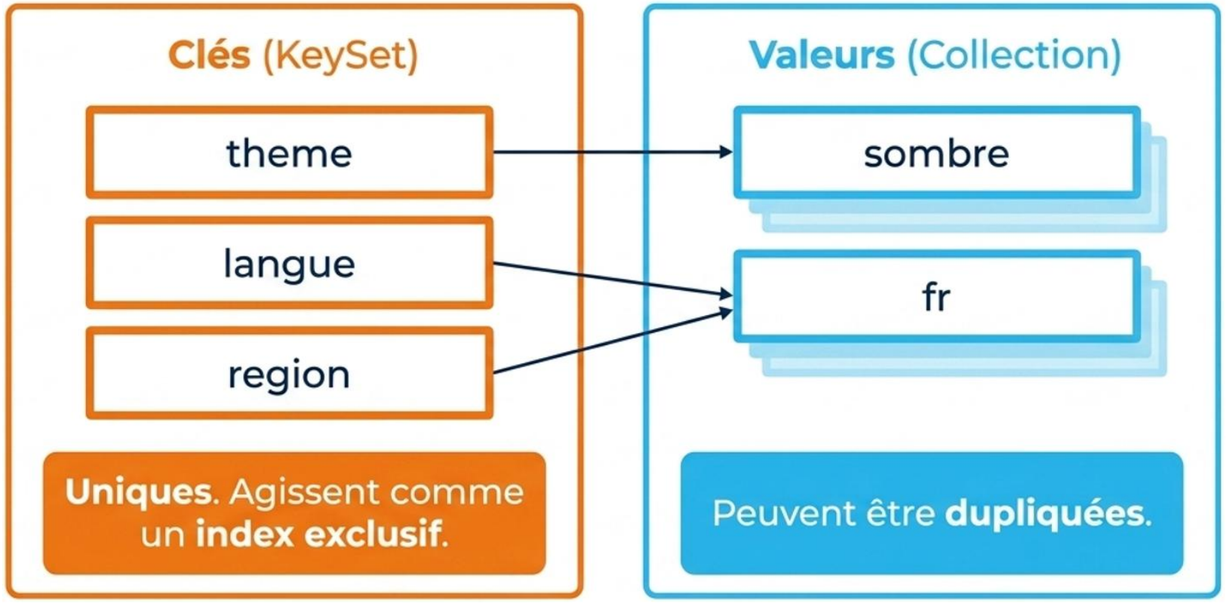
Valeurs Multiples : Les valeurs peuvent être dupliquées (plusieurs clés peuvent pointer vers la même valeur).

Vues de Collection : Méthodes keySet(), values(), et entrySet() pour parcourir les données.

```
Map<String, String> prefs = new HashMap<>();
prefs.put("theme", "sombre");
prefs.put("langue", "fr");
// Clé unique "theme", valeur peut changer
```

Note: La clé est unique, mais sa valeur associée peut être mise à jour ou partagée.

Structure Logique d'une Map



L'interface **Map** n'**hérite PAS** de l'interface **Collection**. Elle modélise une structure d'association mathématique stricte où chaque clé est liée à une seule valeur au maximum.

Méthodes Principales de l'API

API ESSENTIELLE DE L'INTERFACE MAP

Modification

- `put(K, V)` : Ajoute ou remplace une paire clé-valeur.
- `remove(Object k)` : Supprime la paire associée à la clé.
- `clear()` : Vide entièrement la Map.

Accès & Vérification

- `get(Object k)` : Retourne la valeur (ou null si absente).
- `containsKey(Object k)` : Vérifie l'existence d'une clé.
- `size()` / `isEmpty()` : Taille de la collection.

Vues de Collection

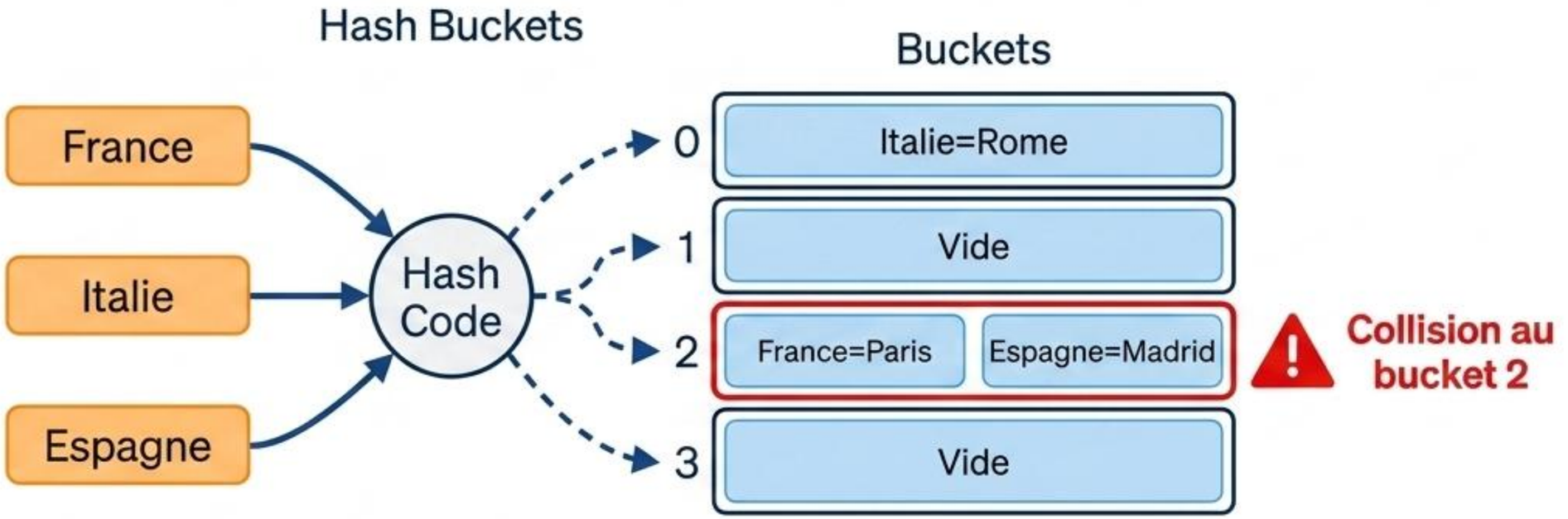
- `keySet()` : Set des clés.
- `values()` : Collection des valeurs.
- `entrySet()` : Set des paires (clé-valeur).

MapMethods.java

```java
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 25);
ages.put("Bob", 30);
ages.put("Alice", 26); // Écrase 25

// Parcours via entrySet (Recommandé)
for (Map.Entry<String, Integer> e : ages.entrySet()) {
  System.out.println(e.getKey() + " -> " + e.getValue());
}
```

HashMap : Table de Hachage



Ordre non garanti : Les éléments sont stockés selon leur hash, pas l'ordre d'insertion.

Haute Performance : Opérations get et put en O(1) en moyenne.

Gestion du Null : Autorise 1 clé null et plusieurs valeurs null.

Non Synchronisée : Non thread-safe par défaut.

HashMap en Pratique : Fréquence des Mots

ALGORITHME DE COMPTAGE DES MOTS

```java
public class WordCounter {
    public static void main(String[] args) {
        String texte = "le chat et le chien jouent le chat dort";
        String[] mots = texte.split(" ");

        Map<String, Integer> compteur = new HashMap<>();

        for (String mot : mots) {
            compteur.put(mot, compteur.getOrDefault(mot, 0) + 1);
        }

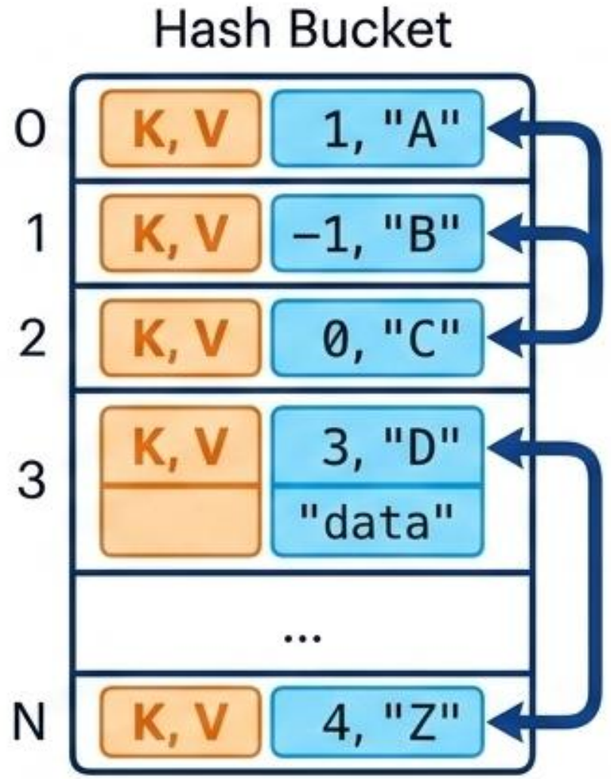
        for (Map.Entry<String, Integer> entry : compteur.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
        // Sortie : { chat : 2, chien : 1, dort : 1, et : 1, jouent : 1, le : 3 }
    }
}
```

getOrDefault() : Simplifie le code en évitant de vérifier manuellement si la clé existe avec containsKey avant d'incrémenter (Java 8+).

Ordre Aléatoire : L'ordre d'affichage n'est pas garanti. Pour trier, il faudrait utiliser TreeMap.

Haute Performance : L'accès aux compteurs se fait en O(1).

LinkedHashMap Préserve la Chronologie



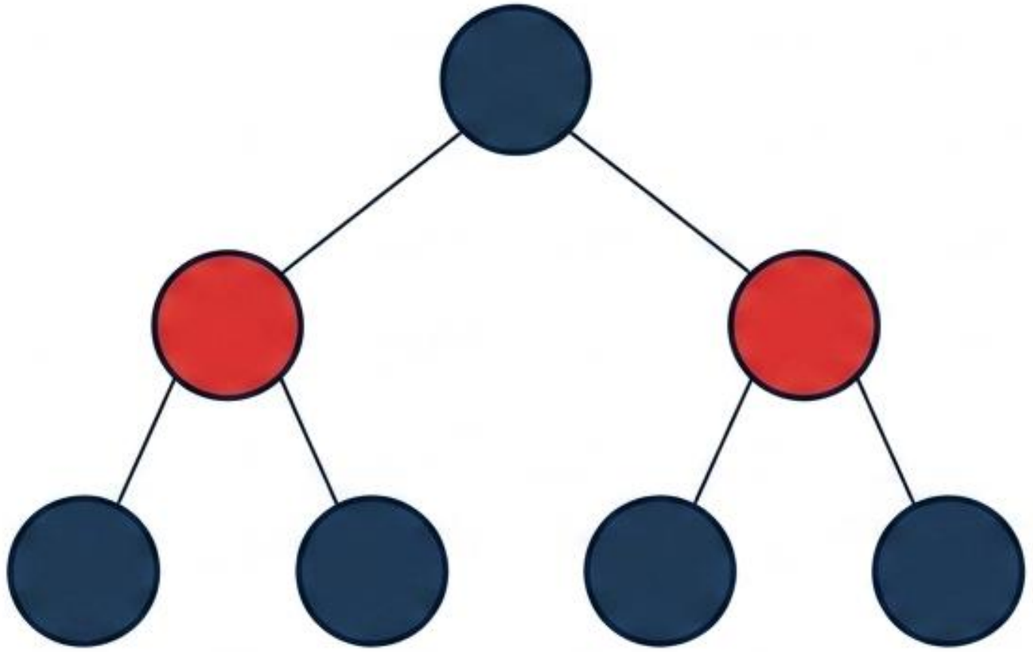

```
cache.put(1, "A");
cache.put(-1, "B");
cache.put(0, "C");
System.out.println(cache.keySet());
```

```
> [1, -1, 0]
```

L'ordre d'insertion est explicitement respecté.

Performance:	Proche de **O(1)**, avec un léger surcoût mémoire pour le chaînage.
Ordre:	Chronologique. Maintient l'ordre d'insertion (ou d'accès).
Cas d'usage:	Parfait pour un cache LRU (Least Recently Used).

TreeMap Impose l'Ordre Naturel



Parcours infixe (gauche-racine-droite) = extraction triée.

Performance:	Complexité **O(log n)** garantie. Plus lent que HashMap, mais systématiquement trié.
Ordre:	Trié selon l'ordre naturel des clés (Comparable) ou un Comparator.
Restriction:	**Clé Null STRICTEMENT INTERDITE** (Provoque NullPointerException).

Personnalisation du Tri dans TreeMap

```java
// 1. Tri inversé (décroissant) simple
TreeMap<Integer, String> scores = new TreeMap<>(Collections.reverseOrder());
scores.put(100, "Or");
scores.put(75, "Platine");
// Sortie : {100=Or, 75=Platine}

// 2. Tri personnalisé avec Lambda (par longueur de chaîne)
Comparator<String> parLongueur = (s1, s2) -> s1.length() - s2.length();
TreeMap<String, Integer> mots = new TreeMap<>(parLongueur);
mots.put("un", 1);
mots.put("trois", 3);
// Sortie : {un=1, trois=3}
```

La méthode la plus simple pour inverser l'ordre naturel. Indispensable pour créer des classements.

Depuis Java 8, définir une logique de tri complexe est concis.

Collections.reverseOrder()

Indispensable pour créer des classements décroissants (tops scores).

Expressions Lambda

Définit une logique de tri concise (depuis Java 8).

Attention aux Doublons

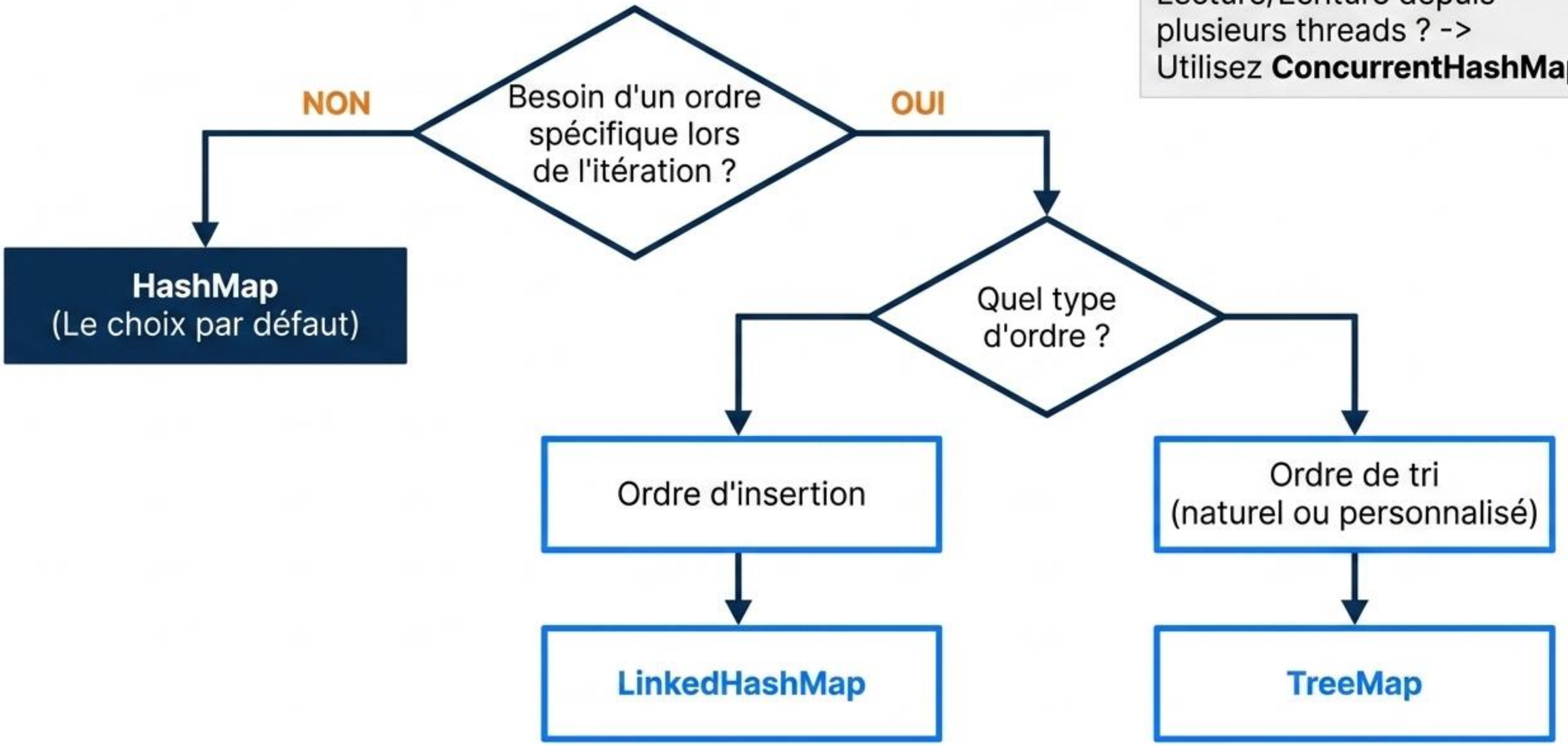
Si le Comparator retourne 0, TreeMap considère les clés égales et écrase la valeur.

Comparatif Stratégique des Implémentations

Caractéristique	HashMap	LinkedHashMap	TreeMap
Complexité	O(1)	O(1)	O(log n)
Ordre d'itération	Aucun garanti	Insertion ou Accès	Trié (Naturel/Comparator)
Clé Null Autorisée ?	Oui (1 seule)	Oui (1 seule)	❌ Non
Structure Interne	Buckets de hachage	Buckets + Liste chaînée	Arbre Rouge-Noir
Cas d'Usage Idéal	Le choix par défaut	Création de caches	Recherches par plages / Tris

L'Arbre de Décision de l'Ingénieur

Exception Multithreading: Lecture/Écriture depuis plusieurs threads ? -> Utilisez **ConcurrentHashMap**.



Standards de Maîtrise pour l'Itération

❌ Le Piège (Double Travail)

```
for (String k : map.keySet()) {
    V v = map.get(k);
}
```

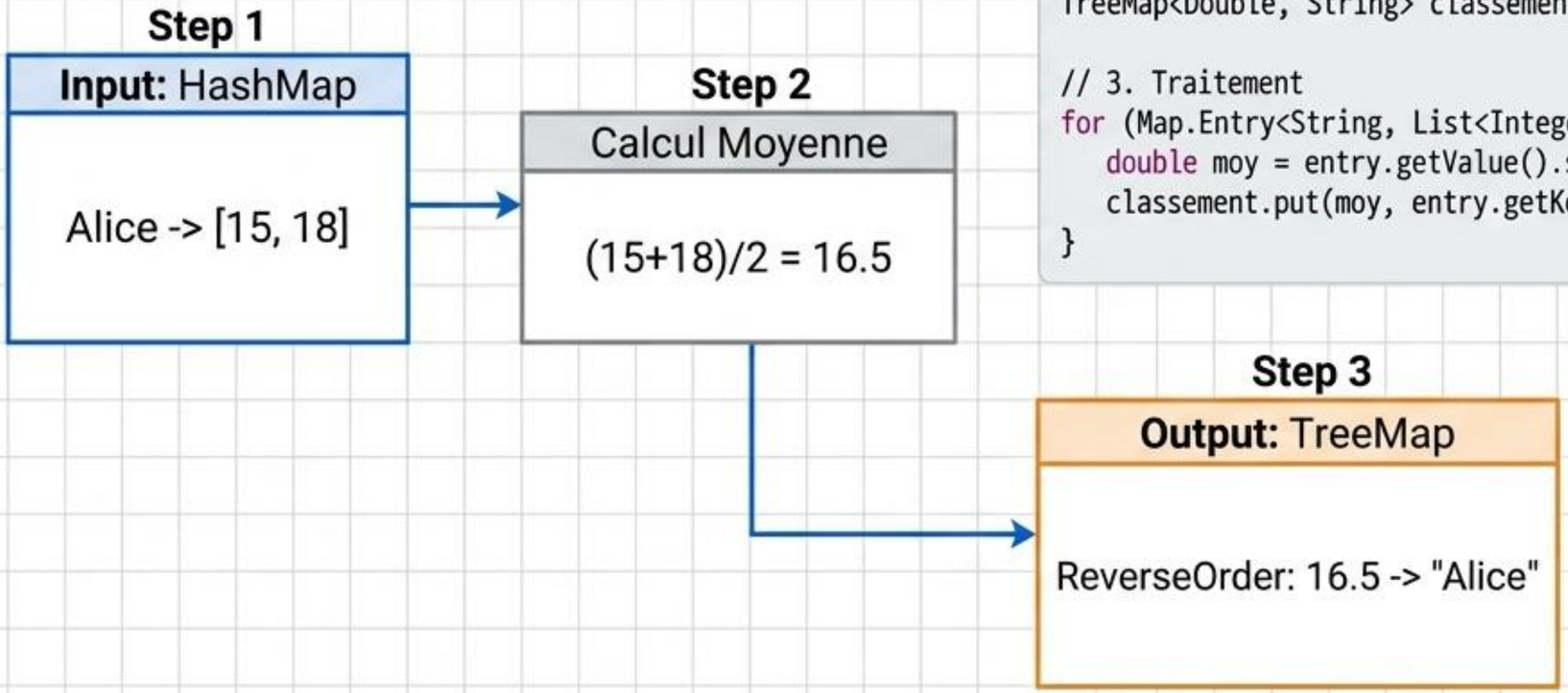
Force un hachage supplémentaire à chaque itération.

✔ La Vue Optimisée (Une seule passe)

```
for (Map.Entry<K,V> entry : map.entrySet()) {
    K key = entry.getKey();
    V value = entry.getValue();
}
```

Extrait simultanément la clé et la valeur en une seule opération mémoire.

Exercice de Synthèse : Pipeline de Classement




```java
// 1. Données brutes (HashMap - Ordre non garanti)
Map<String, List<Integer>> notes = new HashMap<>();
notes.put("Alice", Arrays.asList(15, 18, 16)); // Moy: 16.33

// 2. Préparation du classement (TreeMap - Ordre décroissant)
TreeMap<Double, String> classement = new TreeMap<>(Collections.reverseOrder());

// 3. Traitement
for (Map.Entry<String, List<Integer>> entry : notes.entrySet()) {
   double moy = entry.getValue().stream().mapToInt(a->a).average().orElse(0.0);
   classement.put(moy, entry.getKey()); // Insertion (Tri auto)
}
```

Pièges Fréquents de l'Examen OCA

Clés Immuables

Utilisez exclusivement des objets immuables (String, Integer) comme clés. Modifier l'état d'une clé après insertion détruit l'intégrité de la table de hachage.

L'Arbre Intolérant

Insérer une clé 'null' dans une TreeMap compile parfaitement, mais provoque une NullPointerException fatale à l'exécution en raison de l'impossibilité de comparer dans l'arbre.

La Valeur de Retour Cachée

Attention : put(K, V) ne retourne pas un simple booléen. Elle retourne l'ancienne valeur précédemment associée à cette clé (ou null s'il s'agit d'une nouvelle insertion).

Adoption des Bonnes Pratiques Modernes

BestPractices.java

```java
// Crée la liste si la clé n'existe pas, puis ajoute
index.computeIfAbsent("Java", k -> new ArrayList<>()).add("Chapitre 9");

// Évite la NullPointerException en fournissant une valeur par défaut
List<String> docs = index.getOrDefault("Python", Collections.emptyList());
```

Immuabilité : Clés = Objets immuables (String, Record).

Itération : Toujours utiliser entrySet() au lieu de keySet().

Défaut : Privilégiez toujours HashMap sauf si le tri ou l'ordre chronologique est une exigence métier stricte.